Requirements as the Oracle

Generating REST API Tests from Vague Requirements Without Learning the Bugs They Should Catch

Final report

Team PreTrained

Sohail Memon	2025201075	Mohd Ahmad	2025201029
Kathan Patel	2025201039	Md Taufique Hussain	2025202007

Code and results: `https://github.com/SohailTSM/Requirements-as-the-Oracle`

Abstract

LLM-generated API tests are only useful if they encode the *expected* behaviour. We study two problems on RESTestBench (3 services, 106 requirements, 228 planted bugs) with a single small open model, Llama 3.1 8B. First, requirements in practice are vague. Second, when a refinement loop repairs a test against a possibly buggy service, the test can learn the bug. Our agent retrieves the relevant operations, derives checks, writes a structured plan instead of code, cleans the plan against the specification, compiles it to pytest, runs it and refines it under a guard. On vague requirements it catches 12.7% of planted bugs, against 2.6% for single-step generation ($p < 0.001$). Overall that is on par with RESTestBench's refinement loop (11.3%, n.s.). When tests are refined against the buggy service, which is the realistic case, our agent keeps its score (14.8%) while the refinement loop drops to 4.3% (+10.5 points, 95% CI $[5.6, 15.7]$, $p < 0.001$). The refinement loop's adaptation gap is 7.0 points ($p = 0.004$); ours is indistinguishable from zero for every variant. Ablations show that the deterministic executor (-11.8 points) and specification clean-up (-7.5) carry the gain. The derived checks, retrieval, judge and fine-tuning show no effect beyond run-to-run noise. Two humans labelled 181 items with $\kappa = 0.84$. Against them, the Llama judge used for offline check scoring reaches only $\kappa = 0.03$–0.27, so we report human-labelled precision numbers.

1 Introduction

Crash-oriented REST testing tools (EvoMaster, RESTler, AutoRestTest [2]) use HTTP 5xx or schema violations as the oracle. A requirement violation that returns `200 OK`, such as a user deleting another user's data, is invisible to them. Catching such bugs needs the expected behaviour, which lives in natural-language requirements.

RESTestBench [1] measured how well LLMs turn requirements into tests. Results are good with precise requirements but collapse with vague ones, the realistic case. A refinement loop (run, read the HTTP trace, regenerate) helps, but on a faulty service the model "adapts its assertions to match the faulty behaviour": the refined test encodes the bug as correct. The paper measures this effect but proposes no remedy. LLM-generated oracles are known to capture actual rather than expected behaviour [9].

Our re-analysis of RESTestBench's released files (proposal, §5.1) found that the refinement prompt's few-shot example teaches copying the observed value. Llama 3.1 8B's whole refinement gain on vague requirements (3.1% → 12.3%) disappears when it refines against the buggy service (3.5%).

Target user. Backend and QA engineers who have user stories and an OpenAPI specification, and want tests that check behaviour, cheaply enough to run with a small local model.

2 Literature review

- **Crash/schema fuzzers:** AutoRestTest [2], LlamaRestTest [3], Schemathesis. Their oracle cannot see 200-returning logic bugs. We include Schemathesis as baseline E.
- **Response-judging agents:** LogiAgent [4], ARMeta [5]. The judge sees the response before deciding, so it shares the bias we target. We include a LogiAgent-style judge as baseline B4.
- **Oracle generation without requirements:** SATORI [6], MASTOR [7]. These derive truth from the specification or the implementation.

- **Requirement-based generation:** APITestGenie [8] and RESTestBench's own tools [1]. We include the latter as B1/B2. None of these protects refinement from learning the bug.

3 Research questions and hypotheses

RQ1 (vague requirements). Given only a vague requirement and the OpenAPI specification, can an 8B agent generate tests that catch requirement bugs, and how much of the vague-to-precise gap does it close?

RQ2 (safe refinement). Does restricted refinement remove bug adaptation without raising false alarms on the correct service?

RQ3 (component value). Which components contribute measurably, and which variant of each works best?

H1: deriving implied checks raises the score on vague requirements well above single-step generation. **H2:** with restricted refinement, the adaptation gap is ≈ 0 and false alarms are no higher than with unrestricted refinement. **H3:** tool-based execution sharply reduces tests that fail on the correct service.

4 Dataset

Benchmark used: RESTestBench [1], released with its paper:

- FastAPI (Python), NestJS RealWorld (TypeScript) and TodoApp (.NET), run in Docker;
- 63 / 29 / 14 requirements, each in a vague and a precise wording;
- 141 / 65 / 22 hand-written planted bugs (mutants);
- the unmodified scoring harness.

The agent sees only the requirement text and the specification. FastAPI was the development service; NestJS and TodoApp were held out from every design decision.

Data we created. RESTestBench has no labels below the test level, so we built the following (all in `pretrained/gold/` and `pretrained/finetune/` in the repository):

- **Gold operations** for all 106 requirements: the API operations each reference test calls, parsed from the benchmark's reference tests with the Python AST and matched to the specification. They are the ground truth for retrieval Recall@5 and MRR.
- **Gold checks:** 268 atomic expected-behaviour checks (FastAPI 148, NestJS 94, TodoApp 26), drafted with an AI assistant by splitting the precise requirements, which are human-written. They are the ground truth for the precision layer and the SFT targets. Two humans validated a sample of them (§7.4).
- **Reference plans:** 10 JSON plans for FastAPI requirements, used to test the compiler and runtime.
- **Human labels:** 181 items over 45 requirements (stratified, fixed seed), labelled independently by two team members (§7.4).
- **Fine-tuning data:** SFT examples (vague requirement $\rightarrow$ checks) in leave-one-service-out folds of 86 / 154 / 184 examples; DPO preference pairs from our own refinement histories, with the guarded plan preferred over the plan that copied or changed an expected value to the observed one (1,273 pairs from FastAPI runs; 143 from NestJS and TodoApp, which form the FastAPI-held-out fold, the only one trained).

5 Methodology

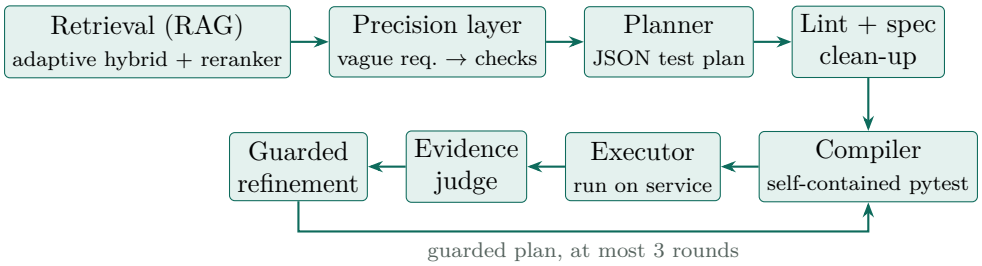


Figure 1: Final pipeline. Everything before the executor uses only the requirement and the specification frozen from the correct service.

(1) **Retrieval (RAG).** Each OpenAPI operation is one chunk (path, method, resolved schemas, description). BM25 and dense embeddings (bge-small) are fused. A cross-encoder (MiniLM) reranks only requirements of at most 30 words, since it hurts long precise ones ("adaptive"). Operations needed for authentication are added by dependency closure.

(2) **Precision layer (reasoning).** The LLM expands the requirement into atomic checks, guided by a catalogue of implied properties: round-trip, echo, ownership, before/after deltas, filter soundness. In the final configuration, 5 sampled drafts are merged (self-consistency). Each check is tagged *meaning* (locked) or *format* (changeable only with specification evidence).

(3) **Plan, not code (tool calling).** The LLM writes a JSON plan (actors, steps, checks). A linter checks it against the specification (unknown operations, references to fields that do not exist, impossible comparisons), and the LLM repairs flagged plans once.

(4) **Specification clean-up.** Deterministic, specification-only fixes:

- drop checks that can never pass;
- generalise status codes the requirement does not state (`200`→`2xx`);
- replace invented credentials;
- redirect misdirected references;
- fill path parameters.

(5) **Compiler and executor (tools).** A compiler turns the plan into a self-contained pytest file. The runtime creates actors with valid credentials, logs in, passes IDs and joins every path to the service base URL.

(6) **Guarded refinement and judge (agentic).** On a failure, the agent decides whether to refine, stop or relax a format check. The rules are:

- Assertion failures are reported without the observed value.
- A guard restores any change to a locked check, and blocks new expected values equal to a value the service just returned.
- Field repairs must name fields the specification documents.
- The judge may blame the test only with evidence from the requirement or the specification. An API_BUG verdict ends the loop.

(7) **Fine-tuning.** QLoRA SFT of the precision layer (one model per held-out service) and DPO preferring "report the bug" over "copy the observed value" (Unsloth + TRL on Kaggle T4s, exported to 4-bit GGUF for Ollama).

We also built and compared these alternatives in a component sweep: a single ReAct agent with tools; multi-persona and blackboard multi-agent designs; tools exposed over an MCP server; reviewer and debugger agents over A2A; and three amounts of content passed between agents (full trace, summary, verdict only).

6 Experiment runs

Model and compute.

- Every LLM role, ours and the baselines', uses Llama 3.1 8B Instruct, 4-bit Q4_K_M, under Ollama: 24k context, temperature 0.2, at most 3 refinement rounds.
- The models were served from free Kaggle T4 GPUs on the four team accounts, through Cloudflare tunnels.
- A local dispatcher queued all requests and failed over between servers: 105,044 requests, 99.8% answered, 251M tokens.
- The services ran on a laptop, with up to 10 Docker copies at once.

Baselines.

B1 RESTestBench single-step generation.

B2 RESTestBench refinement loop, unchanged.

B3 B2 with only the prompts fixed: copying example removed, generator warned, judge feedback passed on, API_BUG outcome added.

B4 LogiAgent-style response judge.

E: the Schemathesis fuzzer, which uses no requirements.

Metrics.

- *Caught*: RESTestBench's score, the % of planted bugs killed by tests generated while interacting with the correct service. A test counts only if it passes on the correct service.
- *Mutant-gen*: the same score for tests generated while interacting with the buggy service, which is the realistic case.
- *Adaptation gap*: Caught − Mutant-gen.
- *Fail-OK*: share of tests failing on the correct service (false alarms).
- Per capability: retrieval Recall@5 and MRR; precision-layer check recall and contradiction rate (human-labelled); planner and lint failure rates; setup-error rate (tool calls); judge verdict accuracy; tampering attempts blocked and accepted; ReAct tool-call validity; cost per test.

Protocol and statistics.

- **Sweep:** one component varied at a time on FastAPI, vague wording, 1 run each.
- **Finals:** the final configuration and 2 runner-ups; all services, both wordings, 3 runs each. Every baseline also ran 3 times.
- **Ablations:** the final configuration minus one component, all services, vague wording, 1 run each.
- **Statistics:**
  - Unit: the requirement ($n = 106$), its score averaged over runs.
  - Paired Wilcoxon signed-rank tests, Holm-corrected within each family, with rank-biserial r.
  - Bootstrap 95% CIs of the pooled difference, resampling requirements.
- **Freeze:** every decision was logged with its reason, and the experiments were frozen on 11 Nov.

7 Evaluation

7.1 RQ1: vague requirements

Tool	Wording	FastAPI	NestJS	TodoApp	**All**	Mutant-gen	Fail-OK
B1 single-step	vague	4.3 ± 1.4	0.0 ± 0.0	0.0 ± 0.0	2.6 ± 0.9	–	97.2
B2 refinement	vague	**17.5** ± 2.3	0.0 ± 0.0	4.5 ± 0.0	11.3 ± 1.4	4.3 ± 2.0	85.2
B3 prompt-fixed	vague	4.5 ± 2.9	0.0 ± 0.0	0.0 ± 0.0	2.8 ± 1.8	3.1 ± 0.9	96.9
B4 response judge	vague	6.6 ± 0.4	3.1 ± 4.1	18.2 ± 0.0	6.7 ± 1.1	–	**40.6**
E fuzzer	–	13.9 ± 7.8	0.0 ± 0.0	9.1 ± 0.0	9.5 ± 4.8	–	70.8
Ours (final)	vague	12.5 ± 3.2	2.6 ± 3.2	**43.9** ± 6.9	**12.7** ± 2.7	**14.8** ± 1.2	70.4
B1 single-step	precise	39.7 ± 2.6	5.1 ± 1.8	1.5 ± 2.6	26.2 ± 1.8	–	77.4
B2 refinement	precise	**47.8** ± 2.2	6.2 ± 2.7	0.0 ± 0.0	**31.3** ± 0.7	**24.8** ± 0.6	68.6
B3 prompt-fixed	precise	37.6 ± 2.5	5.6 ± 4.9	0.0 ± 0.0	24.9 ± 0.3	25.4 ± 3.0	78.0
B4 response judge	precise	13.5 ± 2.5	10.3 ± 5.8	4.5 ± 0.0	11.7 ± 2.9	–	**56.0**
Ours (final)	precise	20.6 ± 5.1	**22.1** ± 3.9	1.5 ± 2.6	19.2 ± 2.2	16.0 ± 3.0	77.7

Table 1: Planted bugs caught (%), mean $\pm$ sd over 3 runs, Llama 3.1 8B for every tool. "–": the tool does not refine against the service, so no mutant-gen score exists.

Ours (final) vs	Δ caught	95% CI	better / worse req.	p (Holm)	r
Vague: caught					
B1 single-step	+10.1	[+5.2, +15.2]	35 / 4	<0.001	+0.75
B2 refinement	+1.5	[−5.3, +7.9]	30 / 22	1.000	+0.09
B3 prompt-fixed	+9.9	[+5.5, +14.6]	33 / 5	<0.001	+0.85
B4 response judge	+6.0	[+1.0, +11.1]	32 / 11	0.032	+0.45
E fuzzer	+3.2	[−2.5, +9.1]	26 / 19	1.000	+0.20
Vague: mutant-gen (refined against the buggy service)					
B2 refinement	**+10.5**	[+5.6, +15.7]	42 / 10	<0.001	+0.69
B3 prompt-fixed	+11.7	[+6.8, +16.8]	45 / 5	<0.001	+0.80
Precise: caught					
B1 single-step	−7.0	[−15.9, +2.2]	26 / 28	1.000	−0.19
B2 refinement	−12.1	[−20.8, −3.5]	20 / 37	0.062	−0.49
B4 response judge	+7.5	[+2.0, +12.8]	30 / 14	0.251	+0.27
Precise: mutant-gen					
B2 refinement	−8.8	[−16.6, −0.9]	32 / 35	1.000	−0.16

Table 2: Paired comparisons per requirement ($n = 106$). Δ = pooled percentage points, ours minus other. Wilcoxon p values are Holm-corrected within each block of comparisons (the full set, including runner-ups, is in `results/tables/stats.md`).

Reading.

- **The pipeline lifts vague requirements well above single-step generation.** It scores +10.1 points over B1 and +9.9 over B3, both $p < 0.001$ with large effects. This closes 35% of the way from B1-vague (2.6) to the best precise score (B2, 31.3).
- **Against B2 it is a draw on the correct service.** The difference is +1.5, CI $[-5.3, +7.9]$.
- **The win comes in the realistic setting.** With refinement against the buggy service, ours keeps 14.8% while B2 falls to 4.3%. That is +10.5 points, 3.4×, $p < 0.001$.
- **The held-out services behave differently from FastAPI.** B2 is strongest on FastAPI. Ours wins on the held-out services: TodoApp vague 43.9 vs at most 18.2; NestJS precise 22.1 vs at

most 10.3. On TodoApp the baselines register users with weak passwords that an undocumented password policy rejects. Our runtime creates valid actors.

- **Weakness: precise wording.** B2 beats ours by 12.1 points (CI excludes 0; Holm $p = 0.062$). Our precision layer and clean-up generalise status codes and drop specific checks, which wastes the extra information a precise requirement gives. Our score barely rises from vague to precise (12.7 → 19.2), while B2's nearly triples.

7.2 RQ2: bug adaptation and safe refinement

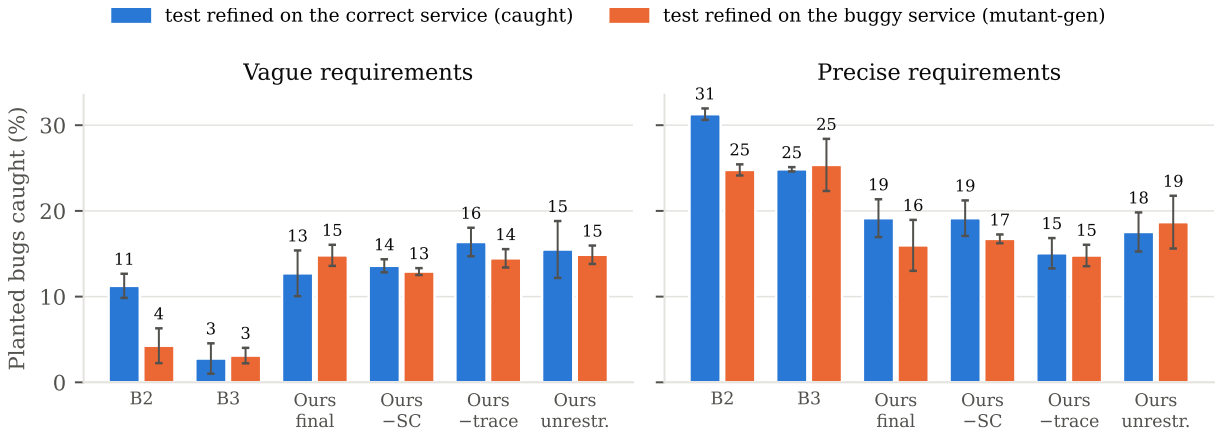


Figure 2: Score of tests refined on the correct service (blue) vs on the buggy service (orange), mean ± sd over 3 runs. B2's tests lose most of their value when they are refined against the buggy service; ours do not. SC = self-consistency.

Tool	Wording	Gap	p	Rounds copying a value	Copies in next plan
B2 refinement	vague	**7.0 ± 2.8**	0.004	261 / 1931 (13.5%)	all
B2 refinement	precise	**6.5 ± 1.3**	0.004	151 / 1825 (8.3%)	all
B3 prompt-fixed	vague	−0.3 ± 1.3	0.86	2 / 33 (6.1%)	all
Ours (final)	vague	−2.1 ± 3.4	0.058	21 / 739 tried (2.8%)	3 / 787 (0.4%)
Ours (final)	precise	3.2 ± 3.0	0.30	57 / 960 tried (5.9%)	8 / 972 (0.8%)
Ours, unrestricted	vague	0.6 ± 3.8	0.68	–	33 / 807 (4.1%)
Ours, unrestricted	precise	−1.1 ± 2.1	0.20	–	135 / 932 (14.5%)

Table 3: Adaptation gap (caught − mutant-gen, mean ± sd over runs) with a per-requirement Wilcoxon test, and value copying in refinement. For ours, "copies reaching the test" compares each next executed plan with the values the service returned. The few residual cases in the final configuration are permitted format changes backed by specification evidence, or values that coincide with the correct behaviour.

- **B2 learns the bugs.** Its gap is 7.0 points on vague and 6.5 on precise wording (both $p = 0.004$). In 8–14% of its refinement rounds it adds an asserted literal taken from the previous response. B3, whose judge almost always answers API_BUG, barely refines (33–74 rounds) and so neither gains nor adapts.
- **Our agent does not, in any variant.** No gap differs from zero, and all mutant-gen differences between our variants are within noise ($|\Delta| \leq 2.7$, all $p > 0.15$).
- **The guard works as a mechanism.** It blocked 512 (vague) and 773 (precise) tampering attempts, mostly attempts to change a locked check (1,196 restored across both wordings). It cut copied values reaching the test from 4.1–14.5% (unrestricted) to under 1%.
- **But the guard is not what removes adaptation here.** The unrestricted variant copies

values yet shows no gap and the same mutant-gen (−0.1, n.s.). Its false-alarm rate is also no different (Fail-OK 68.2 vs 70.4).

- **Where the protection comes from.** It comes mainly from the structure:
  - checks are derived before any execution, so the requirement, not the response, defines them;
  - most refinement rounds repair setup, not expectations (58–70% of rounds have a setup error, §7.3);
  - a copied value in our plans usually equals correct behaviour that a planted bug does not touch.

So H2 is supported for the pipeline, not for the guard alone. The restriction is cheap insurance that removes visible tampering; on this benchmark it shows no measurable effect on scores.

7.3 Intermediate steps, per capability

Capability / step	Metric	Final configuration
RAG: retrieval	Recall@5 / all needed in top 5 / MRR (106 req.)	vague 0.89 / 0.81 / 0.87; precise 0.90 / 0.80 / 0.78 (BM25 vague: 0.79 / 0.69 / 0.70)
Reasoning: precision layer	Gold checks fully covered / soft recall; derived checks contradicting the gold (human labels, 20 req.)	0.29–0.41 / 0.54–0.58; contradiction 0.09 (Llama judge on the same items: 0.30)
Reasoning: planner	Plans that fail to parse after retry; plans with lint issues	2.9% / 51% (vague); 3.1% / 63% (precise)
Tool calling: executor	Rounds with a setup error (missing field reference / setup call rejected)	58% vague (662 / 363), 70% precise (425 / 891)
Agentic: judge	API_BUG verdicts on the correct service, where every failure is the test's fault	117 of 117 (vague), 95 of 96 (precise): no discrimination
Safety: refinement	Tampering attempts blocked / accepted	512 / 0 (vague), 773 / 0 (precise)
Safety: actions	Calls outside the service; forbidden test edits (skip, removed asserts)	0 possible: the runtime joins every path to the base URL (2 plan steps naming another host became invalid requests); 0 (tests are compiled)
Cost	LLM calls / tokens / seconds per generated test	7.7 / 13.9k / 123 s (vague); 8.0 / 16.5k / 183 s (precise)

Table 4: Step-level metrics from the agent histories of the final configuration (3 runs, 1,002 generated tests per wording).

The judge. It answered API_BUG for every failure on the correct service (212 of 213), and returned TEST_BUG only 2 times in 826 verdicts. The 8B judge does not discriminate; it works as a stop rule. This matches the baselines: B3's judge answered API_BUG in 36 of 36 TodoApp rounds. An evidence-restricted judge needs a stronger model than 8B.

7.4 Human validation of the LLM judge

Sohail and Kathan independently labelled 181 items (45 requirements, stratified sample):

- 63 judgements of whether a gold check is correct;
- 118 coverage labels comparing the precision layer's checks with the gold checks.
- **Human–human agreement is high:** Cohen's $\kappa = 0.84$ over 181 paired labels. The task is well defined.
- **The gold data holds up:** 125 of 126 gold-check judgements were "correct" (0.8% wrong).

- **The Llama judge does not agree with humans:** $\kappa = 0.03$ on gold coverage and 0.27 on derived checks. It calls three times as many contradictions (20 vs 6 of 67), calls "partial" coverage "full" (25 of the 102 coverage labels), and answers from the wrong label set in 9 of 118 cases.

Consequence: the offline precision-layer table scored by Llama (Table 5) is unreliable in absolute terms. Only the human-labelled rates are claims. The main results do not use this judge: mutation scores come from the harness.

Precision layer (prompt v2)	Wording	Recall	Contradiction	Checks / req.
catalogue	vague	0.39	0.27	3.6
direct rewrite	vague	0.40	0.21	5.7
self-consistency	vague	0.37	0.26	3.1
catalogue	precise	0.47	0.14	6.8
catalogue, **human labels**	vague	0.29–0.41 (soft 0.54–0.58)	**0.09**	–

Table 5: Offline precision layer against the gold checks (106 requirements), scored by the Llama judge, with the human-labelled sample below.

7.5 Failure analysis

Outcome of the test generated on the correct service (3 runs × 106)	Vague	Precise
Passes and catches at least one planted bug	63 (20%)	61 (19%)
Passes but catches no planted bug (checks too weak)	31 (10%)	10 (3%)
Fails: setup error (wrong field reference, rejected setup call)	85 (27%)	136 (43%)
Fails: a check contradicts the real behaviour; judge blamed the API	112 (35%)	93 (29%)
Fails: a check contradicts the real behaviour; rounds exhausted	12 (4%)	3 (1%)
No valid plan (planner failed)	15 (5%)	15 (5%)

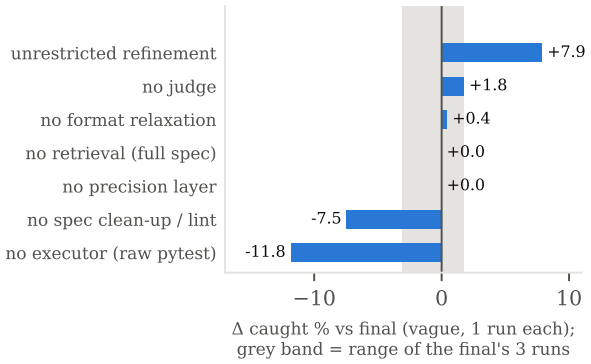
Table 6: Failure taxonomy, computed from the agent histories and the harness results.

The two dominant failures are wrong checks (39% vague) and setup errors (27% vague, 43% precise). Precise requirements name exact payloads, which likely leads the planner into more setup calls the service rejects. Across both wordings, the most frequent setup errors are FastAPI calls that need admin rights (403) and references to request fields that do not exist.

Qualitative examples.

- **Self-contradicting checks** (FastAPI, "Verify that a second user cannot delete an item created by another user"). Self-consistency merged the check "deleting another user's item is rejected (4xx)" with a draft's "… still returns 200". The test can never pass, and the judge blamed the API. A consistency check over derived checks would remove this class.
- **The specification as a misleading oracle** (FastAPI, "Check that log in rejects a wrong password"). The derived check "rejection status is 422, the code documented for this operation" follows the OpenAPI file, but the service returns 400. Specification evidence is not ground truth when the specification is incomplete.
- **The guard at work** (FastAPI mutant, "Delete a non-existing item and confirm the API reports 'Item not found' "). Refinement tried to add a status check equal to the status the buggy service had just returned. The guard blocked it as a copied value, and the locked check kept the test able to kill the mutant.
- **Weak passwords** (TodoApp, all LLM baselines). Generated tests register users with passwords like `password123`, which the service's undocumented policy rejects, so almost every baseline test fails setup. Our runtime generates valid credentials, which explains most of our TodoApp margin.

8 Ablations (RQ3: which components matter)



Removed (vague, 1 run)	All	Fail-OK
final (3 runs)	12.7	70.4
executor (raw pytest)	0.9	98.1
spec clean-up / lint	5.3	85.8
precision layer	12.7	75.5
retrieval (full spec)	12.7	71.7
format relaxation	13.2	74.5
judge	14.5	65.1
restricted refinement	20.6	60.4

Figure 3: Ablations: the final configuration minus one component, all services, vague. The grey band is the range of the final's own 3 runs (9.6–14.5). "Restricted refinement" was re-run 3 times: 15.5 ± 3.3, $\Delta = +2.8$, $p = 0.27$, so its single +7.9 did not replicate.

- **Clear effects: the executor and specification clean-up (support H3).**
  - Without the plan-and-compile executor, 98% of tests fail on the correct service and the score falls to 0.9%. The model writes placeholders such as `Bearer YOUR_TOKEN`.
  - Without clean-up and lint the score falls to 5.3%. The clean-up fires constantly: per 1,002 generated tests (vague) it dropped 443 impossible checks, replaced 253 invented credentials, generalised 252 unstated status codes and redirected 143 references.
- **No measurable effect:** the precision layer, retrieval and format relaxation each leave the score at 12.7–13.2. Removing the judge raises caught slightly (14.5) but lowers mutant-gen (14.8 → 10.4), consistent with the judge acting as an early stop on buggy services. These are single runs inside the noise band.
- **Combining the sweep's winners did not add up.** The final adds two sweep winners (full-trace content +6.4, self-consistency +5.0, single runs) to the default. In the finals, removing either one does not lower the score: the two runner-ups score 0.9 and 3.7 points *higher* than the final, both n.s. The final configuration's own runs spread from 9.6 to 14.5, so single-run differences under about ± 6 points are not readable. We report this as a methodological lesson: the sweep's single runs over-selected noise.
- **Fine-tuning shows no effect beyond noise.** SFT of the precision layer took 23–47 minutes per fold on a T4 (86/154/184 examples, final loss 0.19–0.35). DPO on the FastAPI-held-out fold had 143 pairs, loss 0.69 → 0.14, and took 133 minutes including export. On FastAPI (single sweep runs) the fine-tuned precision layer scored 17.0, SFT as the whole model 15.6 and SFT+DPO 14.2, against the default's 11.3–13.5. All are inside the noise, so the two larger DPO folds were not trained.
- **Architecture.** A single ReAct agent with tools scored 5.0 and a blackboard design 3.5, against 12.0 for the fixed pipeline. Over 2,069 ReAct turns:
  - 66% were well-formed tool calls;
  - 12% wrote the call as plain text (we unwrap and execute these);
  - 22% called no tool at all;
  - 25% of generations ended in the fallback planner.

  The 8B model is not a reliable controller, which is why the final design keeps control in code. MCP (12.1) and A2A (14.9) transport was neutral.

- **4-bit vs fp16.** One fp16 run scored 13.6 against 12.7 ± 2.7 for 4-bit ($\Delta = -0.9$, $p = 0.54$; mutant-gen 14.3 vs 14.8). We found no measurable quantisation penalty, so reporting on 4-bit is justified.

9 Discussion: hypotheses

- **H1: partly supported.** The full pipeline beats single-step generation clearly on vague requirements ($+10.1$, $p < 0.001$). However, the precision layer itself does not account for it: removing it leaves the score unchanged, and human labels show it recovers only 29–41% of gold checks fully. The gain comes from executable structure, not from deriving implied properties.
- **H2: supported for the pipeline, not attributable to the guard.** The adaptation gap is ≈ 0 for every variant of ours, and 7.0 points for B2. False alarms do not rise with restriction. But unrestricted refinement shows the same result, so the plan structure, not the guard, is what protects the tests.
- **H3: supported.** Tool-based execution moves Fail-OK from 98.1% to 70.4% and is the largest single effect (-11.8). Yet 70% of tests still fail on the correct service, so execution reduces false alarms but does not solve them.

10 Limitations and threats to validity

- **Scale.** One benchmark of 3 services. The proposal's optional extension (EMB/RESTgym services) was not done. With 106 requirements and 3 runs, differences below about 5 points are not detectable.
- **Noise.** The final configuration's own runs range from 9.6 to 14.5. Sweep and ablation rows are single runs, and only effects larger than about ± 6 points are interpretable. The choice of the final configuration from single runs was not confirmed by the finals.
- **Development service.** All design decisions were made on FastAPI. The held-out services favour us, but the clean-up rules (e.g. credential replacement) encode lessons learned while building on FastAPI.
- **One model.** Every result is for Llama 3.1 8B 4-bit. The planned GPT-5 Nano reference was dropped to keep the study to one open model. Conclusions about the judge and the ReAct agent may not hold for larger models.
- **Fine-tuning.** Fine-tuning was tested only in single sweep runs on FastAPI, with one DPO fold.
- **The LLM judge.** The judge used for offline check scoring was shown to be unreliable; only its human-validated sample supports claims.

11 Conclusion and future work

With a single 8B model, structure beats cleverness. A plan-compile-execute pipeline with specification clean-up lifts vague-requirement testing from 2.6% to 12.7%. It also stays robust when refined against a buggy service, where RESTestBench's refinement loop loses two-thirds of its score (14.8% vs 4.3%). The components that ask the 8B model to reason more did not measurably help: derived checks, an evidence judge, a ReAct controller and fine-tuning. Its judgements, both as the evidence judge and as the offline scorer, did not survive validation.

Future work:

- a consistency check over derived checks;
- a stronger, validated judge;
- using precise requirements without generalising their details away;
- replicating on more services with more runs per configuration.

Acknowledgment

We thank our course instructor, Prof. Vasudeva Varma, and the TAs for their guidance, the RESTestBench authors for releasing their benchmark, and Kaggle for free GPU time.

A Deviations from the proposal

- **4-bit instead of fp16 for the reported runs:** one fp16 run confirms no measurable difference.
- **No GPT-5 Nano reference:** single-model study (Llama only).
- **DPO trained on one fold only:** the larger folds were dropped because the first showed no gain.
- **The ablation "minus fine-tuning" is replaced by the sweep's fine-tuned rows:** the final configuration uses the base model.
- **The planned paired Wilcoxon tests use the requirement as the unit.**

B Reproducibility

- **Repository:** `https://github.com/SohailTSM/Requirements-as-the-Oracle`. Its `README.md` gives the setup, the commands to run one job or a whole queue, and the commands that rebuild every table and figure.
- **Code:** the agent (`pretrained/agent/`), baselines (`tools_ext/`), the queue runner and dispatcher (`scripts/`), fine-tuning jobs (`finetune/`) and server notebooks (`kaggle_llm_server/`) are in the repository.
- **Configurations:** every configuration is a YAML file in `configs/agent/`.
- **Results:** every reported number is regenerated from the result files by `make_tables.py`, `final_stats.py` and `make_figures.py`.
- **Decisions:** every design decision and its reason is recorded in `Plan/Decision_Log.md`.

References

[1] L. Kogler, S. Hangler, M. Ehrhart, B. Dornauer, R. Wuersching, P. Schrammel. *RESTestBench: A Benchmark for Evaluating the Effectiveness of LLM-Generated REST API Test Cases from NL Requirements.* EASE 2026. arXiv:2604.25862.

[2] M. Kim et al. *A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs (AutoRestTest).* ICSE 2025. arXiv:2411.07098.

[3] M. Kim et al. *LlamaRestTest: Effective REST API Testing with Small Language Models.* FSE 2025. arXiv:2501.08598.

[4] *LogiAgent: Automated Logical Testing for REST Systems with LLM-Based Multi-Agents.* arXiv:2503.15079, 2025.

[5] *Multi-Agent LLM-based Metamorphic Testing for REST APIs (ARMeta).* arXiv:2605.28321, 2026.

[6] J. C. Alonso et al. *SATORI: Static Test Oracle Generation for REST APIs.* ASE 2025. arXiv:2508.16318.

[7] *MASTOR: A Multi-Agent Approach to Semantic Test Oracle Generation for RESTful APIs.* arXiv:2606.10465, 2026.

[8] *APITestGenie: Generating Web API Tests from Requirements and API Specifications with LLMs.* arXiv:2604.02039, 2026.

[9] M. Konstantinou et al. *Do LLMs Generate Test Oracles that Capture the Actual or the Expected Program Behaviour?* arXiv:2410.21136, 2024.